State Monad

State · Pure RNG / bank balance simulation · Build with Waqar

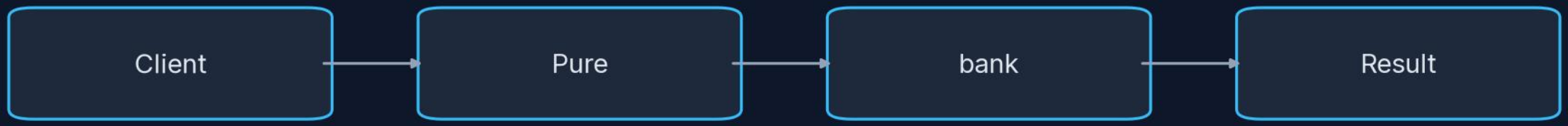
Today's Hook

- Problem: Pure RNG / bank balance simulation
- Pattern: State — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Pure RNG / bank balance simulation
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



State: Pure RNG / bank balance simulation

Real-World Scenario

- Running example: Pure RNG / bank balance simulation
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```
package com.buildwithwaqar.patterns.functional.state.problem;

import java.util.*;

public final class NaiveRng {
  private int seed; NaiveRng(int s){ seed=s; }
  public int next(){ seed = seed*1103515245+12345; return Math.abs(seed%100); }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.functional.state.problem;

import java.util.*;
import java.util.function.*;

record State<S,A>(Function<S, Map.Entry<A,S>> runF){
  Map.Entry<A,S> run(S s){ return runF.apply(s); }
  static <S,A> State<S,A> pure(A a){ return new State<>(s -> Map.entry(a,s)); }
  <B> State<S,B> map(Function<A,B> f){ return flatMap(a -> State.pure(f.apply(a))); }
  <B> State<S,B> flatMap(Function<A, State<S,B>> f){
    return new State<>(s -> { var r=runF.apply(s); return f.apply(r.getKey()).runF.apply(r.getValue()); });
  }
  static State<Integer,Integer> nextInt(){ return new State<>(s -> { int n=Math.abs((s*1103515245+12345)%100); return Map.entry(n, n==0?1:n); }); }
  static State<Integer,Integer> deposit(int cents){ return new State<>(bal -> Map.entry(bal+cents, bal+cents)); }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.functional.state.problem;
import java.util.*; import java.util.function.*;
public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== State: pure RNG / bank balance ===");
    System.out.println("--- naive ---"); var n=new NaiveRng(1); System.out.println(n.next()+" "+n.next());
System.out.println("--- state RNG ---");
var rng=State.nextInt().flatMap(a -> State.nextInt().map(b -> List.of(a,b)));
System.out.println(rng.run(1));
System.out.println("--- state bank ---");
System.out.println(State.deposit(50).flatMap(b -> State.deposit(25)).run(0));
  }
}
```

Walkthrough Output

```
=== State: pure RNG / bank balance ===
--- naive ---
90 73
--- state RNG ---
[90, 87]=87
--- state bank ---
75=75
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with State.

Related Patterns

- Contrast State with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → State → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.